

-- =====

-- RETAILMART — SQL CAPSTONE PROJECT

-- Industry : Retail (3-store chain)

-- Duration : 3 Hours

-- Covers : DDL, DML, SELECT, JOINS, GROUP BY, Subqueries,

-- UNION/INTERSECT/EXCEPT, CTEs,

-- Window Functions, PIVOT, Views, Indexes,

-- Stored Procedures

-- =====

-- STORY:

-- You are a data analyst at RetailMart — a retail chain with

-- 3 stores across Pakistan. Your job is to build their

-- database from scratch and answer key business questions.

-- ===== ASSIGNMENT_7 =====

Name : NAHEED DEEDAR

ID : 801925

-- =====

-- HOUR 1 — BUILD & LOAD

-- =====

-- =====

-- TASK 1: DDL — Create the schema

-- =====

-- =====

-- RETAILMART — CREATE TABLES PRACTICE

-- Task: Fill in the blanks to complete each CREATE TABLE.

-- Table names, column names, and hints are given.

-- You must choose the correct data type and constraints.

-- =====

-- DATA TYPES TO CHOOSE FROM:

-- INT, VARCHAR(n), CHAR(n), DECIMAL(10,2), DATE

--

-- CONSTRAINTS TO CHOOSE FROM:

-- PRIMARY KEY, IDENTITY(1,1), NOT NULL, UNIQUE,

-- DEFAULT, CHECK, REFERENCES

-- =====

create schema retailmart;

go

-- =====

-- TABLE 1: categories

-- Stores product categories like Electronics, Clothing, etc.

-- =====

CREATE TABLE retailmart.categories (

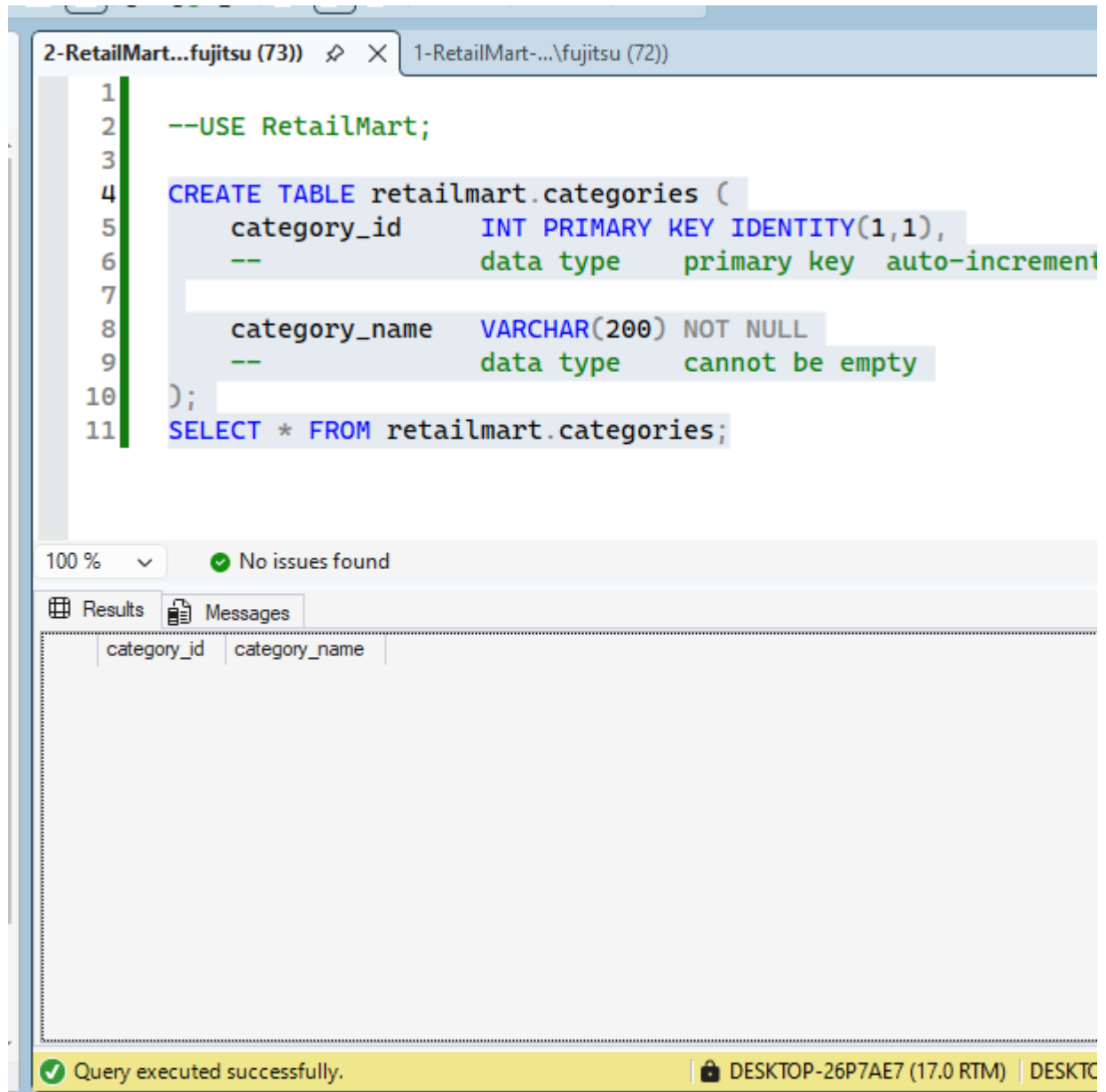
category_id INT PRIMARY KEY IDENTITY(1,1),

-- data type primary key auto-increment

category_name VARCHAR(200) NOT NULL

-- data type cannot be empty

);



2-RetailMart...fujitsu (73)) 1-RetailMart-...\fujitsu (72))

```
1
2  --USE RetailMart;
3
4  CREATE TABLE retailmart.categories (
5      category_id      INT PRIMARY KEY IDENTITY(1,1),
6      --               data type    primary key  auto-increment
7
8      category_name    VARCHAR(200) NOT NULL
9      --               data type    cannot be empty
10 );
11 SELECT * FROM retailmart.categories;
```

100 % No issues found

Results Messages

category_id	category_name
-------------	---------------

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP

-- =====

-- TABLE 2: products

-- Stores all products sold at RetailMart.

-- =====

CREATE TABLE retailmart.products (

product_id INT PRIMARY KEY IDENTITY(1,1),

-- data type primary key auto-increment

product_name VARCHAR(300) NOT NULL,

-- data type cannot be empty

category_id INT NOT NULL REFERENCES retailmart.categories(category_id),

-- data type cannot be links to categories table

-- empty

price DECIMAL(10,2) NOT NULL CHECK(PRICE >0),

-- decimal type cannot be must be greater than 0

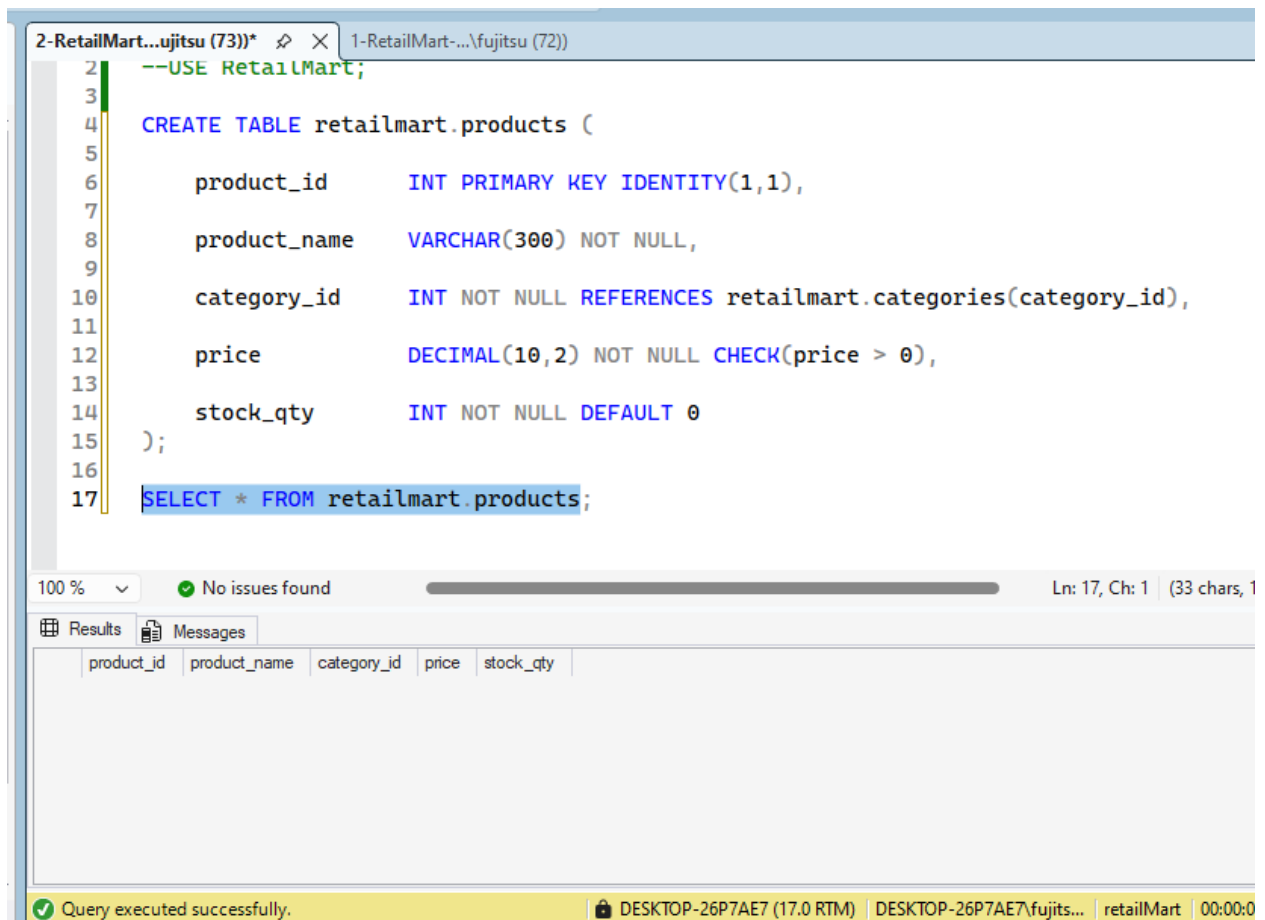
-- empty

stock_qty INT NOT NULL default 0

-- data type cannot be default value = 0

-- empty

);



```
-- =====
-- TABLE 3: stores
-- The 3 RetailMart store locations.
-- =====
```

```
CREATE TABLE retailmart.stores (
```

```
store_id INT PRIMARY KEY IDENTITY(1,1),
```

```
-- data type primary key auto-increment
```

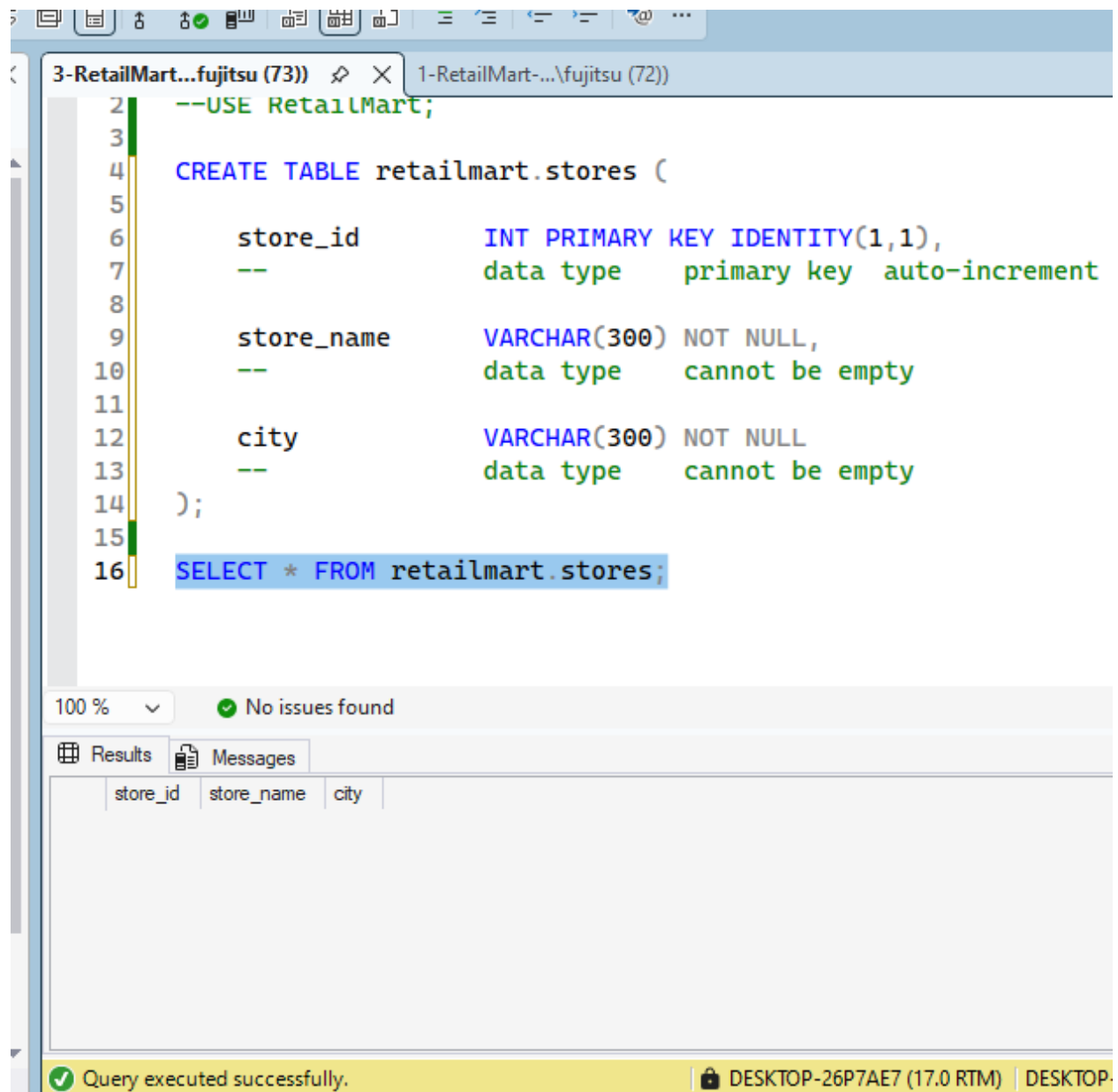
```
store_name VARCHAR(300) NOT NULL,
```

```
--      data type  cannot be empty

city      VARCHAR(300) NOT NULL,

--      data type  cannot be empty

);
```



```
2  --USE RetailMart;
3
4  CREATE TABLE retailmart.stores (
5
6      store_id      INT PRIMARY KEY IDENTITY(1,1),
7      --            data type  primary key  auto-increment
8
9      store_name     VARCHAR(300) NOT NULL,
10     --            data type  cannot be empty
11
12     city            VARCHAR(300) NOT NULL
13     --            data type  cannot be empty
14 );
15
16 SELECT * FROM retailmart.stores;
```

100 % No issues found

Results Messages

store_id	store_name	city
----------	------------	------

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-

```
-- =====
```

-- TABLE 4: customers

-- Registered customers of RetailMart.

-- =====

CREATE TABLE retailmart.customers (

customer_id INT PRIMARY KEY IDENTITY(1,1),

-- data type primary key auto-increment

full_name VARCHAR(300) NOT NULL,

-- data type cannot be empty

email VARCHAR(300) NOT NULL UNIQUE,

-- data type cannot be no two customers

-- empty can share email

city VARCHAR(300),

-- data type (nullable - customer may not provide city)

gender CHAR(1) CHECK(gender IN ('M', 'F'))

-- single char only 'M' or 'F' allowed

);

```
4-RetailMart...fujitsu (73) 1-RetailMart-...fujitsu (72)
3
4 CREATE TABLE retailmart.customers (
5
6     customer_id    INT PRIMARY KEY IDENTITY(1,1),
7     --             data type    primary key    auto-increment
8
9     full_name      VARCHAR(300) NOT NULL,
10    --             data type    cannot be empty
11
12    email           VARCHAR(300) NOT NULL UNIQUE,
13    --             data type    cannot be      no two customers
14    --             empty        can share email
15
16    city            VARCHAR(300),
17    --             data type (nullable - customer may not provide city)
18
19    gender           CHAR(1) CHECK(gender IN ('M', 'F'))
20    --             single char only 'M' or 'F' allowed
21 );
22
23 SELECT * FROM retailmart.customers;
```

100 % No issues found

Results Messages

customer_id	full_name	email	city	gender
-------------	-----------	-------	------	--------

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7fujitsu... retailMart

-- =====

-- TABLE 5: orders

-- Each row = one order placed by a customer at a store.

-- =====

CREATE TABLE retailmart.orders (

order_id INT PRIMARY KEY IDENTITY(1,1),

-- data type primary key auto-increment


```

store_id    INT NOT NULL REFERENCES retailmart.stores(store_id),
--          data type    cannot be    links to stores table
--
--          empty

customer_id INT NOT NULL REFERENCES retailmart.customers(customer_id),
--          data type    cannot be    links to customers table
--
--          empty

order_date  DATE NOT NULL DEFAULT GETDATE(),
--          date type    cannot be    default = today's date
--
--          empty        (hint: use GETDATE())

status      VARCHAR(200) NOT NULL DEFAULT 'Pending' CHECK(STATUS IN ('Pending' , 'Shipped' ,
'Delivered' , 'Cancelled'))
--          data type    cannot be    default =    only allow:
--
--          empty        'Pending'  'Pending'
--
--                      'Shipped'
--
--                      'Delivered'
--
--                      'Cancelled'
);

```

```

12
13     customer_id    INT NOT NULL REFERENCES retailmart.customers(customer_id),
14     --            data type    cannot be    links to customers table
15     --
16
17     order_date     DATE NOT NULL DEFAULT GETDATE(),
18     --            date type    cannot be    default = today's date
19     --            empty        (hint: use GETDATE())
20
21     status         VARCHAR(200) NOT NULL DEFAULT 'Pending' CHECK(status IN ('Pending' , 'Shipped'
22     --            data type    cannot be    default =    only allow:
23     --            empty        'Pending'    'Pending'
24     --                        'Shipped'
25     --                        'Delivered'
26     --                        'Cancelled'
27 );
28
29
30
31 SELECT * FROM retailmart.orders;

```

100 % No issues found Ln: 31, Ch: 1 (32 chars, 1 lines) SPC CRLF UTF

order_id	store_id	customer_id	order_date	status
----------	----------	-------------	------------	--------

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... retailMart 00:00:00 Row: 1, Col: 1 0 rows

```

-- =====
-- TABLE 6: order_items
-- Each row = one product line inside an order.
-- One order can have multiple items.
-- =====

```

CREATE TABLE retailmart.order_items (

item_id INT PRIMARY KEY IDENTITY(1,1),

-- data type primary key auto-increment

order_id INT NOT NULL REFERENCES retailmart.orders(order_id),

```
--      data type  cannot be  links to orders table
--
--      empty

product_id  INT NOT NULL REFERENCES retailmart.products(product_id),
--      data type  cannot be  links to products table
--
--      empty

quantity    INT NOT NULL CHECK(quantity >0),
--      data type  cannot be  must be greater than 0
--
--      empty

unit_price  DECIMAL(10,2) NOT NULL
--      decimal type cannot be empty
);
```

```
6-RetailMart...fujitsu (73)) 1-RetailMart-...fujitsu (72))
10      --      data type      cannot be      links to orders table
11      --
12
13      product_id      INT NOT NULL REFERENCES retailmart.products(product_id),
14      --      data type      cannot be      links to products table
15      --      empty
16
17      quantity      INT NOT NULL CHECK(quantity >0),
18      --      data type      cannot be      must be greater than 0
19      --      empty
20
21      unit_price      DECIMAL(10,2) NOT NULL
22      --      decimal type cannot be empty
23  );
24
25
26
27
28
29  SELECT * FROM retailmart.order_items;
```

100 % 1 0 1 0 Ln: 29, Ch: 1 (37 chars, 1)

item_id	order_id	product_id	quantity	unit_price
---------	----------	------------	----------	------------

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... retailMart 00:00:00

-- END OF EXERCISE

-- =====

-- =====

-- TASK 2: DML — INSERT, UPDATE, DELETE, MERGE

-- =====

-- INSERT categories

INSERT INTO retailmart.categories (category_name) VALUES

('Electronics'), ('Clothing'), ('Groceries'), ('Accessories'), ('Home Appliances');

The screenshot shows a SQL IDE with two tabs: '8-RetailMart...ujitsu (73)*' and '1-RetailMart-...fujitsu (72)'. The active tab displays the following SQL code:

```
1  
2  --USE RetailMart;  
3  
4  INSERT INTO retailmart.categories (category_name) VALUES  
5  ('Electronics'), ('Clothing'), ('Groceries'), ('Accessories'), ('Home Appliances');  
6  
7  
8  
9  SELECT * FROM retailmart.categories;
```

Below the code editor, the 'Results' tab is active, showing a table with 5 rows and 2 columns: 'category_id' and 'category_name'.

	category_id	category_name
1	1	Electronics
2	2	Clothing
3	3	Groceries
4	4	Accessories
5	5	Home Appla...

The status bar at the bottom indicates 'Query executed successfully.' and provides system information: 'DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | retailMart | 00:00:00 | Row: 1'.

-- INSERT products

INSERT INTO retailmart.products (product_name, category_id, price, stock_qty) VALUES

('Samsung 4K TV', 1, 85000, 15),

('iPhone 15', 1, 295000, 10),

('Wireless Earbuds', 4, 4500, 80),

('Leather Jacket', 2, 8500, 40),

('Linen Shirt', 2, 2200, 60),

('Rice 5kg', 3, 950, 200),

('Cooking Oil 3L', 3, 750, 150),

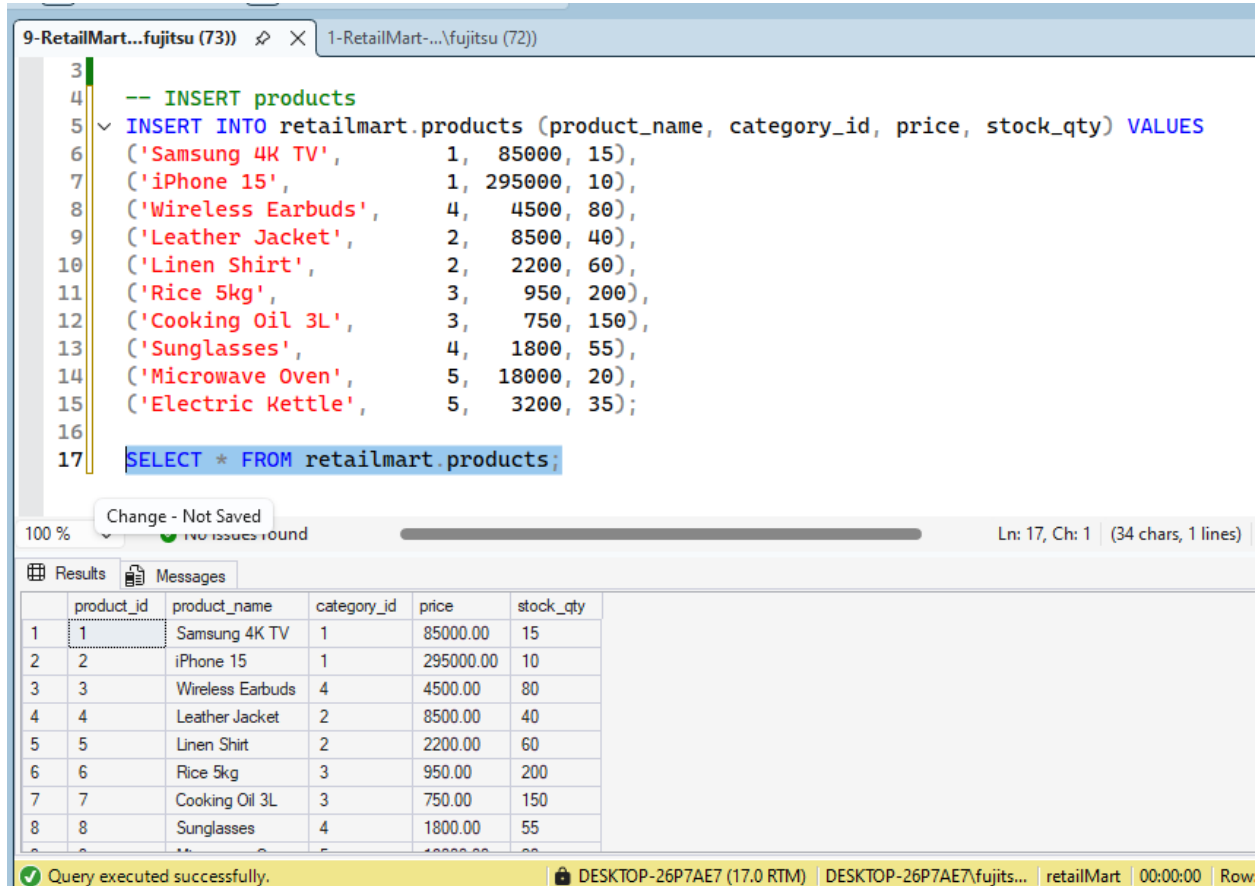
```

('Sunglasses',      4, 1800, 55),

('Microwave Oven',   5, 18000, 20),

('Electric Kettle',   5, 3200, 35);

```



The screenshot shows a SQL IDE with two tabs: '9-RetailMart...fujitsu (73)' and '1-RetailMart-...fujitsu (72)'. The active tab contains the following SQL script:

```

3
4  -- INSERT products
5  INSERT INTO retailmart.products (product_name, category_id, price, stock_qty) VALUES
6  ('Samsung 4K TV',      1, 85000, 15),
7  ('iPhone 15',          1, 295000, 10),
8  ('Wireless Earbuds',   4, 4500, 80),
9  ('Leather Jacket',     2, 8500, 40),
10 ('Linen Shirt',        2, 2200, 60),
11 ('Rice 5kg',           3, 950, 200),
12 ('Cooking Oil 3L',     3, 750, 150),
13 ('Sunglasses',         4, 1800, 55),
14 ('Microwave Oven',     5, 18000, 20),
15 ('Electric Kettle',    5, 3200, 35);
16
17 SELECT * FROM retailmart.products;

```

The IDE interface includes a status bar at the bottom showing 'Change - Not Saved', '100 %', 'Ln: 17, Ch: 1 (34 chars, 1 lines)', and a message 'Query executed successfully.'.

The 'Results' tab displays the output of the SELECT query as a table with 8 rows and 6 columns:

	product_id	product_name	category_id	price	stock_qty
1	1	Samsung 4K TV	1	85000.00	15
2	2	iPhone 15	1	295000.00	10
3	3	Wireless Earbuds	4	4500.00	80
4	4	Leather Jacket	2	8500.00	40
5	5	Linen Shirt	2	2200.00	60
6	6	Rice 5kg	3	950.00	200
7	7	Cooking Oil 3L	3	750.00	150
8	8	Sunglasses	4	1800.00	55

-- INSERT stores

```
INSERT INTO retailmart.stores (store_name, city) VALUES
```

```
('RetailMart Karachi', 'Karachi'),
```

```
('RetailMart Lahore', 'Lahore'),
```

```
('RetailMart Islamabad', 'Islamabad');
```

10-RetailMar...ujitsu (73))* 1-RetailMart-...fujitsu (72))

```
1  
2 --USE RetailMart;  
3  
4 -- INSERT stores  
5 INSERT INTO retailmart.stores (store_name, city) VALUES  
6 ('RetailMart Karachi', 'Karachi'),  
7 ('RetailMart Lahore', 'Lahore'),  
8 ('RetailMart Islamabad', 'Islamabad');  
9  
10  
11  
12  
13 SELECT * FROM retailmart.stores;
```

100 % No issues found

Results Messages

	store_id	store_name	city
1	1	RetailMart Karachi	Karachi
2	2	RetailMart Lahore	Lahore
3	3	RetailMart Islamabad	Islamabad

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26

-- INSERT customers

INSERT INTO retailmart.customers (full_name, email, city, gender) VALUES

('Ahmed Khan', 'ahmed@gmail.com', 'Karachi', 'M'),

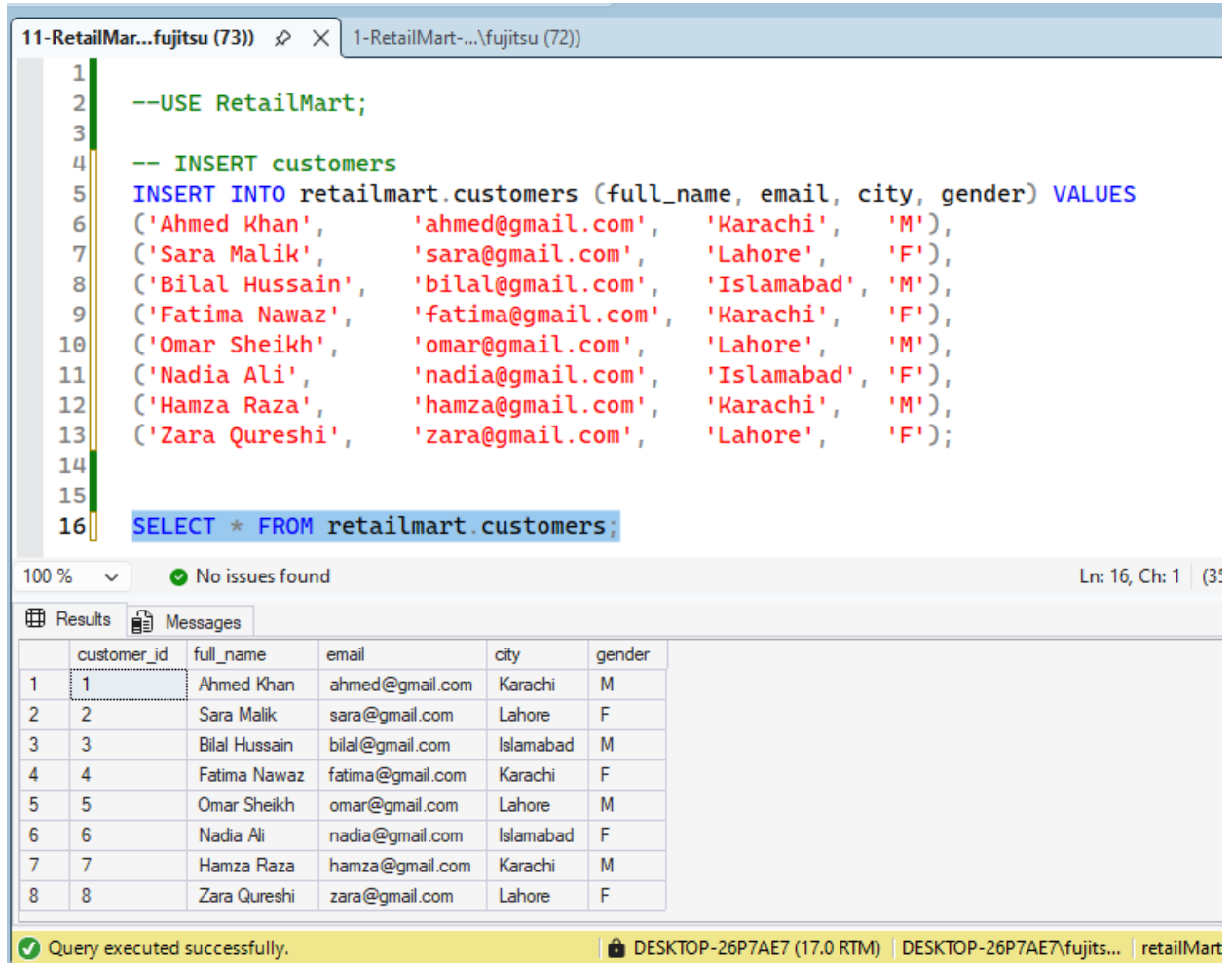
('Sara Malik', 'sara@gmail.com', 'Lahore', 'F'),

('Bilal Hussain', 'bilal@gmail.com', 'Islamabad', 'M'),

('Fatima Nawaz', 'fatima@gmail.com', 'Karachi', 'F'),

('Omar Sheikh', 'omar@gmail.com', 'Lahore', 'M'),

```
('Nadia Ali', 'nadia@gmail.com', 'Islamabad', 'F'),  
('Hamza Raza', 'hamza@gmail.com', 'Karachi', 'M'),  
('Zara Qureshi', 'zara@gmail.com', 'Lahore', 'F');
```



```
1  
2 --USE RetailMart;  
3  
4 -- INSERT customers  
5 INSERT INTO retailmart.customers (full_name, email, city, gender) VALUES  
6 ('Ahmed Khan', 'ahmed@gmail.com', 'Karachi', 'M'),  
7 ('Sara Malik', 'sara@gmail.com', 'Lahore', 'F'),  
8 ('Bilal Hussain', 'bilal@gmail.com', 'Islamabad', 'M'),  
9 ('Fatima Nawaz', 'fatima@gmail.com', 'Karachi', 'F'),  
10 ('Omar Sheikh', 'omar@gmail.com', 'Lahore', 'M'),  
11 ('Nadia Ali', 'nadia@gmail.com', 'Islamabad', 'F'),  
12 ('Hamza Raza', 'hamza@gmail.com', 'Karachi', 'M'),  
13 ('Zara Qureshi', 'zara@gmail.com', 'Lahore', 'F');  
14  
15  
16 SELECT * FROM retailmart.customers;
```

100 % No issues found Ln: 16, Ch: 1 (3!

Results Messages

	customer_id	full_name	email	city	gender
1	1	Ahmed Khan	ahmed@gmail.com	Karachi	M
2	2	Sara Malik	sara@gmail.com	Lahore	F
3	3	Bilal Hussain	bilal@gmail.com	Islamabad	M
4	4	Fatima Nawaz	fatima@gmail.com	Karachi	F
5	5	Omar Sheikh	omar@gmail.com	Lahore	M
6	6	Nadia Ali	nadia@gmail.com	Islamabad	F
7	7	Hamza Raza	hamza@gmail.com	Karachi	M
8	8	Zara Qureshi	zara@gmail.com	Lahore	F

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujits... retailMart

-- INSERT orders

```
INSERT INTO retailmart.orders (store_id, customer_id, order_date, status) VALUES
```

```
(1, 1, '2024-01-10', 'Delivered'),
```

```
(1, 4, '2024-01-15', 'Delivered'),
```

```
(2, 2, '2024-02-03', 'Delivered'),
```

```
(2, 5, '2024-02-20', 'Shipped'),
```

```
(3, 3, '2024-03-05', 'Delivered'),
```



```

(3, 6, '2024-03-18', 'Pending'),
(1, 7, '2024-04-01', 'Delivered'),
(2, 8, '2024-04-22', 'Cancelled'),
(1, 1, '2024-05-10', 'Delivered'),
(3, 3, '2024-06-01', 'Shipped');

```

The screenshot shows a SQL Server query window with the following SQL code:

```

-- INSERT orders
INSERT INTO retailmart.orders (store_id, customer_id, order_date, status) VALUES
(1, 1, '2024-01-10', 'Delivered'),
(1, 4, '2024-01-15', 'Delivered'),
(2, 2, '2024-02-03', 'Delivered'),
(2, 5, '2024-02-20', 'Shipped'),
(3, 3, '2024-03-05', 'Delivered'),
(3, 6, '2024-03-18', 'Pending'),
(1, 7, '2024-04-01', 'Delivered'),
(2, 8, '2024-04-22', 'Cancelled'),
(1, 1, '2024-05-10', 'Delivered'),
(3, 3, '2024-06-01', 'Shipped');

SELECT * FROM retailmart.orders;

```

The query was executed successfully. The results pane shows the following data:

order_id	store_id	customer_id	order_date	status
1	1	1	2024-01-10	Delivered
2	1	4	2024-01-15	Delivered
3	2	2	2024-02-03	Delivered
4	2	5	2024-02-20	Shipped
5	3	3	2024-03-05	Delivered
6	3	6	2024-03-18	Pending
7	1	7	2024-04-01	Delivered
8	2	8	2024-04-22	Cancelled

The status bar indicates: Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujits... | retailMart | 00:00:

-- INSERT order items

```
INSERT INTO retailmart.order_items (order_id, product_id, quantity, unit_price) VALUES
```

```

(1, 2, 1, 295000),
(1, 3, 2, 4500),
(2, 4, 1, 8500),
(3, 5, 3, 2200),

```

```

(3, 8, 1, 1800),
(4, 1, 1, 85000),
(5, 9, 1, 18000),
(5, 10, 2, 3200),
(6, 6, 4, 950),
(7, 3, 1, 4500),
(8, 2, 1, 295000),
(9, 4, 2, 8500),
(10, 9, 1, 18000);

```

The screenshot shows a SQL IDE window with a query editor and a results pane. The query editor contains an INSERT statement into the `retailmart.order_items` table. The results pane displays the data inserted, with columns `item_id`, `order_id`, `product_id`, `quantity`, and `unit_price`.

Query:

```

INSERT INTO retailmart.order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 2, 1, 295000),
(1, 3, 2, 4500),
(2, 4, 1, 8500),
(3, 5, 3, 2200),
(3, 8, 1, 1800),
(4, 1, 1, 85000),
(5, 9, 1, 18000),
(5, 10, 2, 3200),
(6, 6, 4, 950),
(7, 3, 1, 4500),
(8, 2, 1, 295000),
(9, 4, 2, 8500),
(10, 9, 1, 18000);

```

Results:

	item_id	order_id	product_id	quantity	unit_price
1	1	1	2	1	295000.00
2	2	1	3	2	4500.00
3	3	2	4	1	8500.00
4	4	3	5	3	2200.00
5	5	3	8	1	1800.00
6	6	4	1	1	85000.00
7	7	5	9	1	18000.00
8	8	5	10	2	3200.00

Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | retailMart | 00:00:00 | Row: 1,

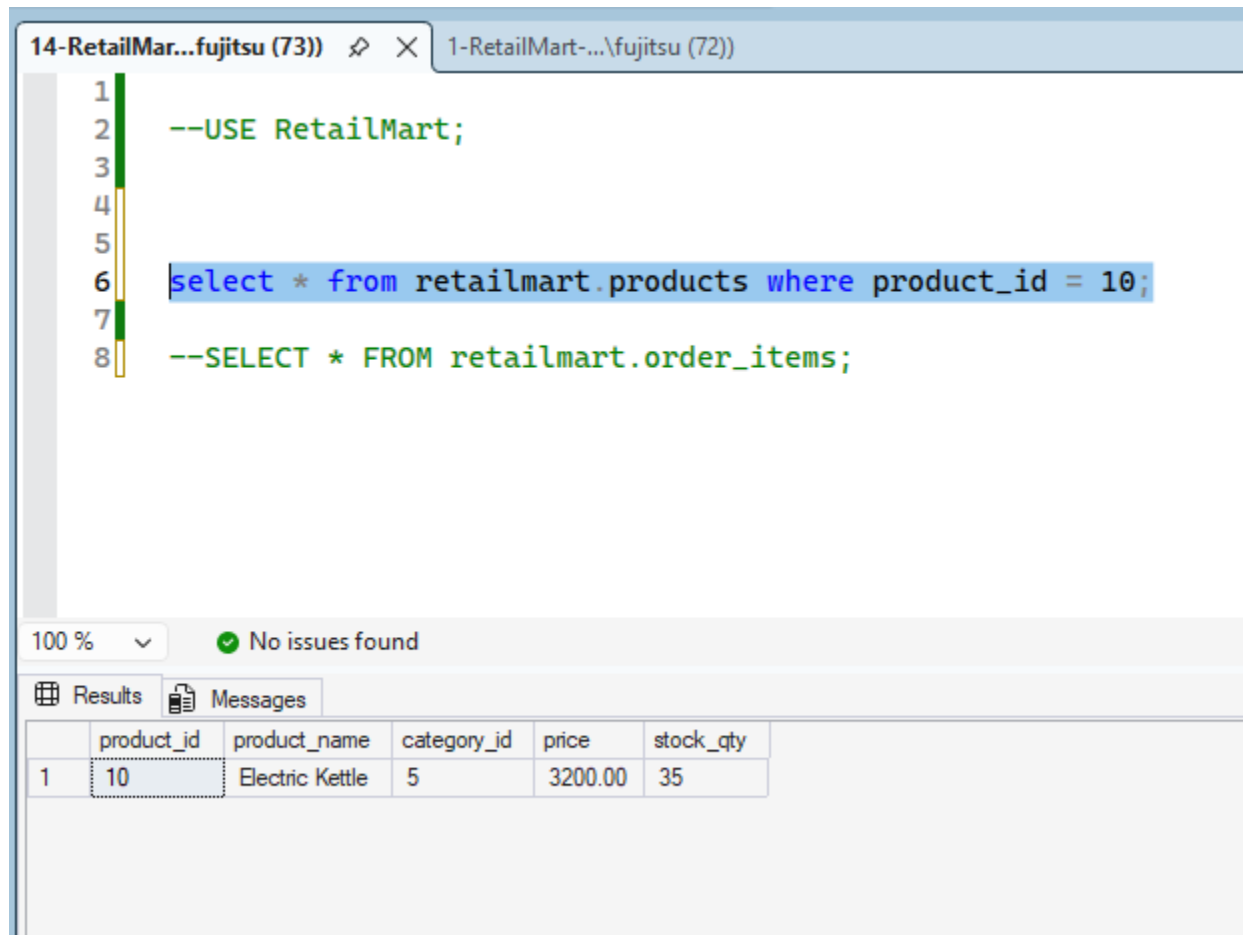
-- UPDATE: Reduce price of Electric Kettle by 10%

```
select * from retailmart.products where product_id = 10;
```

```
UPDATE retailmart.products
```

SET price = price * 0.90

WHERE product_id = 10;



The screenshot shows a SQL IDE interface. At the top, there are two tabs: "14-RetailMar...fujitsu (73))" and "1-RetailMart-...\fujitsu (72))". The active tab displays a SQL script with line numbers 1 through 8. The script contains the following text:

```
1  
2  --USE RetailMart;  
3  
4  
5  
6  select * from retailmart.products where product_id = 10;  
7  
8  --SELECT * FROM retailmart.order_items;
```

Below the script editor, there is a status bar showing "100 %" and "No issues found". Underneath, there are two tabs: "Results" and "Messages". The "Results" tab is active and displays a table with the following data:

	product_id	product_name	category_id	price	stock_qty
1	10	Electric Kettle	5	3200.00	35

The screenshot shows a SQL IDE window with two tabs: '14-RetailMar...ujitsu (73))*' and '1-RetailMart-...\fujitsu (72))'. The active tab contains the following SQL script:

```
1
2  --USE RetailMart;
3
4
5  -- Before Update Electric Kettle Price -- 3200.00
6  select * from retailmart.products where product_id = 10;
7  --After Update: 2880.00
8  -- UPDATE: Reduce price of Electric Kettle by 10%
9  select * from retailmart.products where product_id = 10;
10 UPDATE retailmart.products
11 SET price = price * 0.90
12 WHERE product_id = 10;
13
14
```

Below the script, the status bar indicates '100 %' and 'No issues found'. The 'Results' tab is active, displaying a table with the following data:

	product_id	product_name	category_id	price	stock_qty
1	10	Electric Kettle	5	2880.00	35

-- DELETE: Remove the cancelled order's items and the order itself

DELETE FROM retailmart.order_items WHERE order_id = 8;

DELETE FROM retailmart.orders WHERE order_id = 8;

```
1
2  --USE RetailMart;
3
4  SELECT * FROM retailmart.order_items
5  WHERE order_id = 8;
6  SELECT * FROM retailmart.orders
7  WHERE order_id = 8;
8
9  -- DELETE: Remove the cancelled order's items and the order it
10 DELETE FROM retailmart.order_items WHERE order_id = 8;
11 DELETE FROM retailmart.orders      WHERE order_id = 8;
12
13
14
```

100 %

No issues found

Results

Messages

	item_id	order_id	product_id	quantity	unit_price
1	11	8	2	1	295000.00

	order_id	store_id	customer_id	order_date	status
1	8	2	8	2024-04-22	Cancelled

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-2

```
1
2  --USE RetailMart;
3
4  SELECT * FROM retailmart.order_items
5  WHERE order_id = 8;
6  SELECT * FROM retailmart.orders
7  WHERE order_id = 8;
8
9  -- DELETE: Remove the cancelled order's items and the order itself
10 DELETE FROM retailmart.order_items WHERE order_id = 8;
11 DELETE FROM retailmart.orders      WHERE order_id = 8;
12
13
14
```

100 %

No issues found

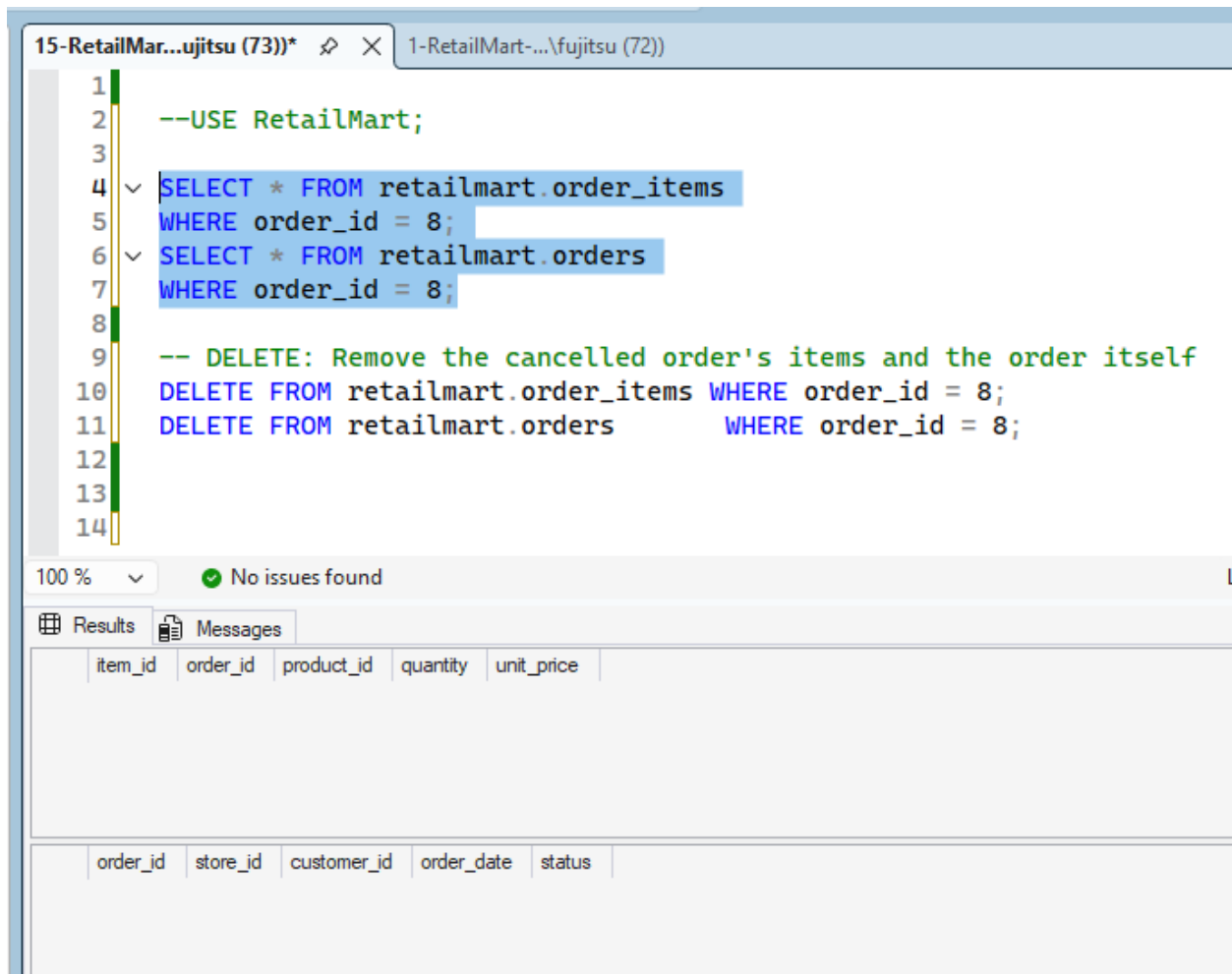
Ln: 9, Ch: 1 (179 c)

Messages

(1 row affected)

(1 row affected)

Completion time: 2026-06-07T11:04:59.7477354+05:00



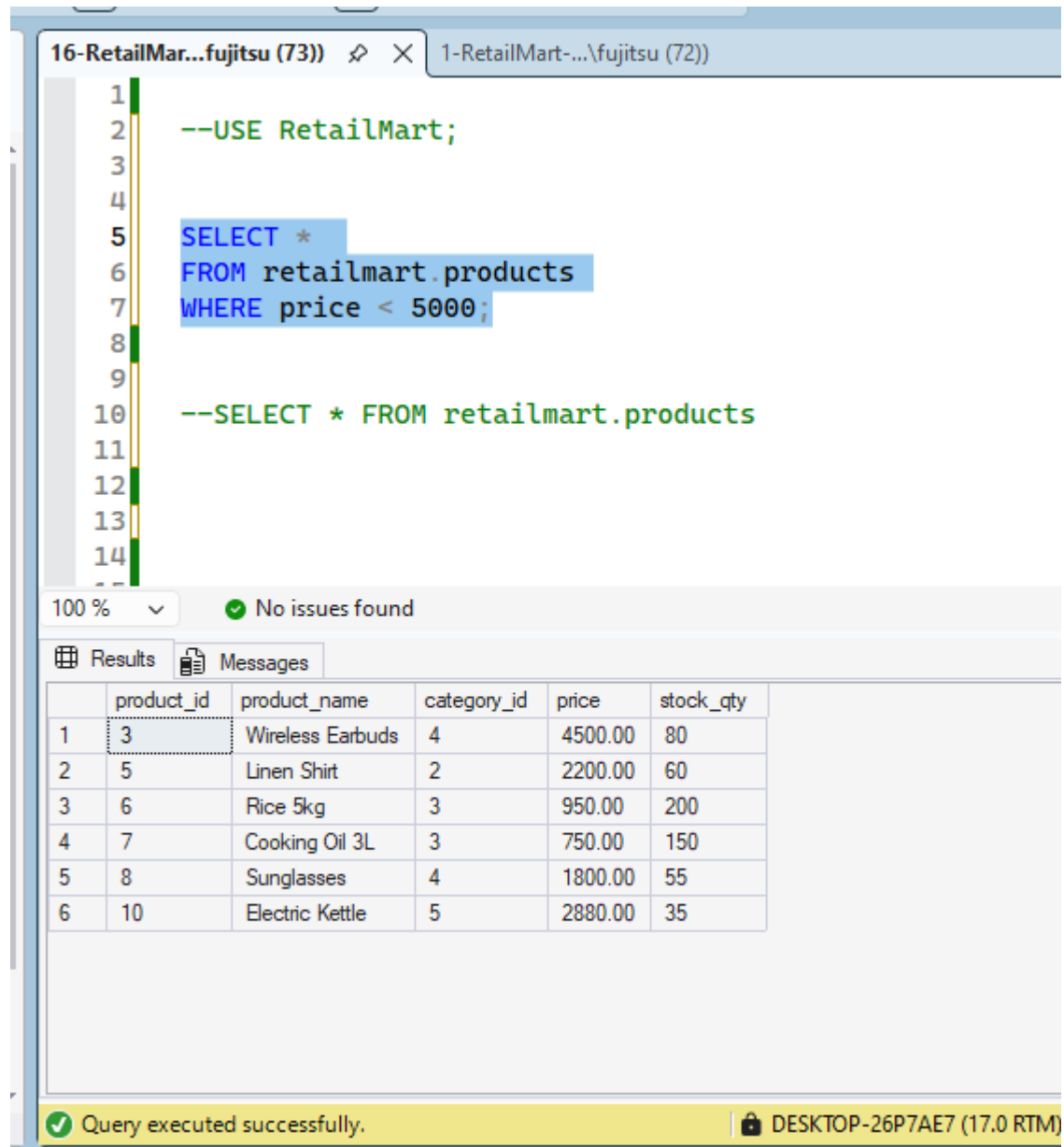
```
-- =====
-- HOUR 2 — QUERY & ANALYZE
-- =====

-- =====
-- TASK 3: SELECT, filtering, sorting
-- =====

-- All products under Rs. 5,000
```

```
select * from retailmart.products
```

```
where price < 5000;
```



16-RetailMar...fujitsu (73)) 1-RetailMart-...\fujitsu (72))

```
1
2  --USE RetailMart;
3
4
5  SELECT *
6  FROM retailmart.products
7  WHERE price < 5000;
8
9
10 --SELECT * FROM retailmart.products
11
12
13
14
```

100 % No issues found

Results Messages

	product_id	product_name	category_id	price	stock_qty
1	3	Wireless Earbuds	4	4500.00	80
2	5	Linen Shirt	2	2200.00	60
3	6	Rice 5kg	3	950.00	200
4	7	Cooking Oil 3L	3	750.00	150
5	8	Sunglasses	4	1800.00	55
6	10	Electric Kettle	5	2880.00	35

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM)

```
-- Customers from Karachi
```

```
select * from [retailmart].[customers]
```

```
where city = 'Karachi';
```

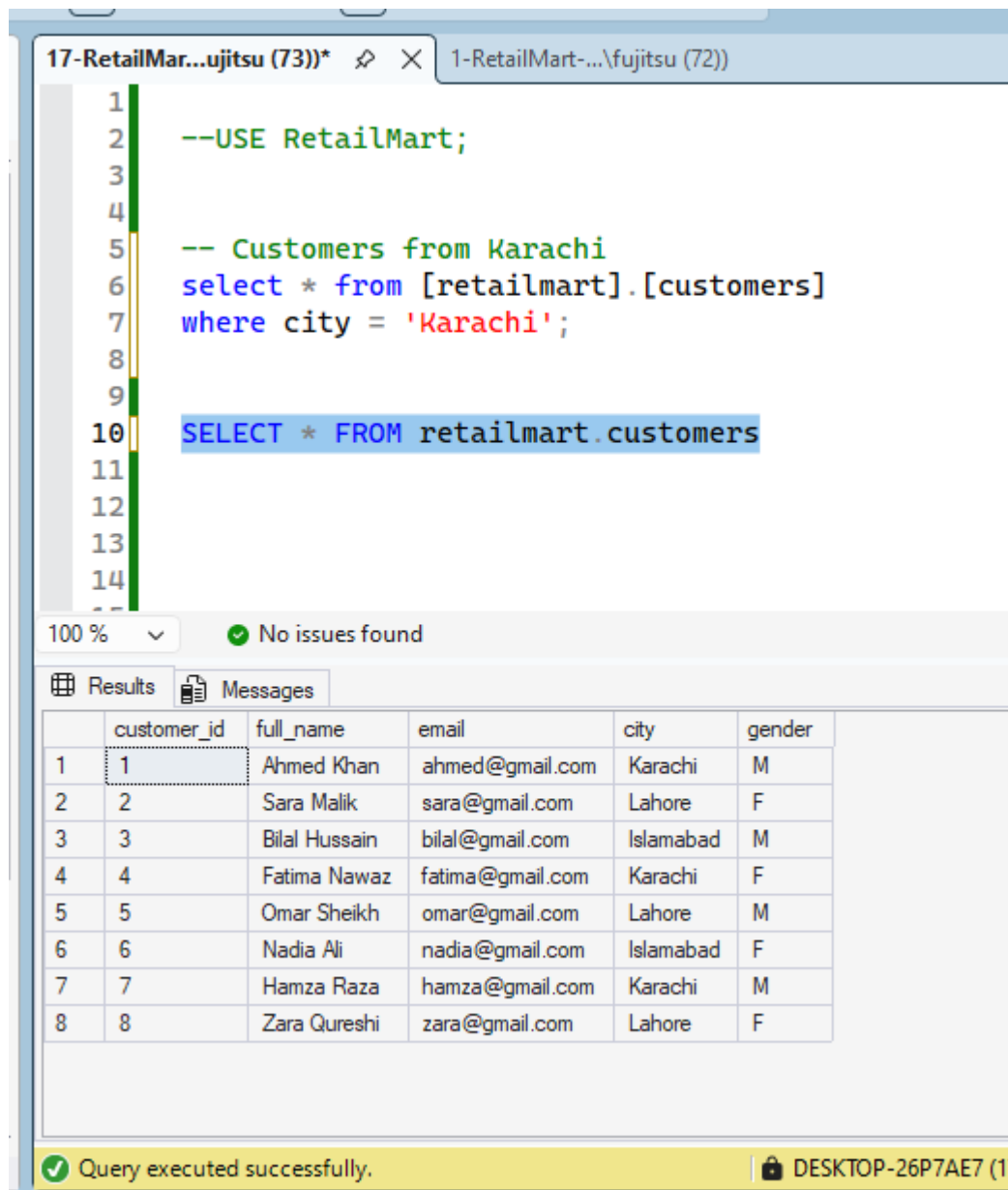

17-RetailMar...ujitsu (73))* 1-RetailMart-...\\fujitsu (72))

```
1
2  --USE RetailMart;
3
4
5  -- Customers from Karachi
6  select * from [retailmart].[customers]
7  where city = 'Karachi';
8
9
10 --SELECT * FROM retailmart.products
11
12
13
14
--
```

100 % No issues found

Results Messages

	customer_id	full_name	email	city	gender
1	1	Ahmed Khan	ahmed@gmail.com	Karachi	M
2	4	Fatima Nawaz	fatima@gmail.com	Karachi	F
3	7	Hamza Raza	hamza@gmail.com	Karachi	M



17-RetailMar...ujitsu (73))* 1-RetailMart-...\fujitsu (72))

```
1
2 --USE RetailMart;
3
4
5 -- Customers from Karachi
6 select * from [retailmart].[customers]
7 where city = 'Karachi';
8
9
10 SELECT * FROM retailmart.customers
11
12
13
14
```

100 % No issues found

Results Messages

	customer_id	full_name	email	city	gender
1	1	Ahmed Khan	ahmed@gmail.com	Karachi	M
2	2	Sara Malik	sara@gmail.com	Lahore	F
3	3	Bilal Hussain	bilal@gmail.com	Islamabad	M
4	4	Fatima Nawaz	fatima@gmail.com	Karachi	F
5	5	Omar Sheikh	omar@gmail.com	Lahore	M
6	6	Nadia Ali	nadia@gmail.com	Islamabad	F
7	7	Hamza Raza	hamza@gmail.com	Karachi	M
8	8	Zara Qureshi	zara@gmail.com	Lahore	F

Query executed successfully. DESKTOP-26P7AE7 (1

-- Orders placed in 2024 sorted by date descending

```
select * from [retailmart].[orders]
```

```
where year(order_date) = '2024';
```

18-RetailMar...fujitsu (73)

1-RetailMart-...\fujitsu (72)

```

1
2  --USE RetailMart;
3
4
5  -- Orders placed in 2024 sorted by date descending
6  SELECT *
7  FROM retailmart.orders
8  WHERE YEAR(order_date) = 2024
9  ORDER BY order_date DESC;
10
11
12
13  --SELECT * FROM retailmart.customers
14

```

100 %

✓ No issues found

Results

Messages

	order_id	store_id	customer_id	order_date	status
1	10	3	3	2024-06-01	Shipped
2	9	1	1	2024-05-10	Delivered
3	7	1	7	2024-04-01	Delivered
4	6	3	6	2024-03-18	Pending
5	5	3	3	2024-03-05	Delivered
6	4	2	5	2024-02-20	Shipped
7	3	2	2	2024-02-03	Delivered
8	2	1	4	2024-01-15	Delivered
9	1	1	1	2024-01-10	Delivered

✓ Query executed successfully.

DESKTOP-26P7AE7 (17.0)

-- =====

-- TASK 4: JOINS — INNER, LEFT, RIGHT

-- =====

-- INNER JOIN: Orders with customer name and product details

select

```
c.customer_id,  
c.full_name,  
o.order_id,  
o.order_date,  
p.product_name  
from [retailmart].[customers] c  
inner join [retailmart].[orders] o  
on c.customer_id=o.customer_id  
inner join [retailmart].[order_items] oi  
on oi.order_id=o.order_id  
inner join [retailmart].[products] p
```

on p.product_id=oi.product_id;

The screenshot shows a SQL query editor with a query window titled "18-RetailMar...ujitsu (73))*" and a results window titled "1-RetailMart-...\fujitsu (72))". The query is a SELECT statement with the following SQL code:

```
5 SELECT
6     c.customer_id,
7     c.full_name,
8     o.order_id,
9     o.order_date,
10    p.product_name
11 FROM retailmart.customers c
12 INNER JOIN retailmart.orders o
13     ON c.customer_id = o.customer_id
14 INNER JOIN retailmart.order_items oi
15     ON o.order_id = oi.order_id
16 INNER JOIN retailmart.products p
17     ON oi.product_id = p.product_id;
18
```

Below the query, the status bar indicates "100 % No issues found". The results window shows a table with 11 rows and 6 columns: customer_id, full_name, order_id, order_date, and product_name. The data is as follows:

	customer_id	full_name	order_id	order_date	product_name
1	1	Ahmed Khan	1	2024-01-10	iPhone 15
2	1	Ahmed Khan	1	2024-01-10	Wireless Earbuds
3	4	Fatima Nawaz	2	2024-01-15	Leather Jacket
4	2	Sara Malik	3	2024-02-03	Linen Shirt
5	2	Sara Malik	3	2024-02-03	Sunglasses
6	5	Omar Sheikh	4	2024-02-20	Samsung 4K TV
7	3	Bilal Hussain	5	2024-03-05	Microwave Oven
8	3	Bilal Hussain	5	2024-03-05	Electric Kettle
9	6	Nadia Ali	6	2024-03-18	Rice 5kg
10	7	Hamza Raza	7	2024-04-01	Wireless Earbuds
11	1	Ahmed Khan	9	2024-05-10	Leather Jacket

At the bottom, a status bar indicates "Query executed successfully." and the system information "DESKTOP-26P7AE7 (17.0 RTM) | DES".

-- LEFT JOIN: All customers including those with no orders

20-RetailMar...fujitsu (73)) 1-RetailMart-...\fujitsu (72))

```
4
5  -- LEFT JOIN: All customers including those with no orders
6
7  SELECT
8      c.customer_id,
9      c.full_name,
10     o.order_id,
11     o.order_date
12 FROM retailmart.customers c
13 LEFT JOIN retailmart.orders o
14     ON c.customer_id = o.customer_id;
15
16 --SELECT * FROM retailmart.customers;
17 --SELECT * FROM retailmart.orders;
```

100 % No issues found

Results Messages

	customer_id	full_name	order_id	order_date
1	1	Ahmed Khan	1	2024-01-10
2	1	Ahmed Khan	9	2024-05-10
3	2	Sara Malik	3	2024-02-03
4	3	Bilal Hussain	5	2024-03-05
5	3	Bilal Hussain	10	2024-06-01
6	4	Fatima Nawaz	2	2024-01-15
7	5	Omar Sheikh	4	2024-02-20
8	6	Nadia Ali	6	2024-03-18
9	7	Hamza Raza	7	2024-04-01
10	8	Zara Qureshi	NULL	NULL

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7

-- RIGHT JOIN: All products including those never ordered

21-RetailMar...fujitsu (73))

1-RetailMart-...\fujitsu (72))

```

6
7  -- RIGHT JOIN: All products including those never ordered
8
9  SELECT
10     p.product_id,
11     p.product_name,
12     oi.order_id,
13     oi.quantity
14 FROM retailmart.order_items oi
15 RIGHT JOIN retailmart.products p
16     ON oi.product_id = p.product_id;
17
18 --SELECT * FROM retailmart.order_items;
19 --SELECT * FROM retailmart.orders;

```

100 %

No issues found

Results

Messages

	product_id	product_name	order_id	quantity
1	1	Samsung 4K TV	4	1
2	2	iPhone 15	1	1
3	3	Wireless Earbuds	1	2
4	3	Wireless Earbuds	7	1
5	4	Leather Jacket	2	1
6	4	Leather Jacket	9	2
7	5	Linen Shirt	3	3
8	6	Rice 5kg	6	4
9	7	Cooking Oil 3L	NULL	NULL
10	8	Sunglasses	3	1
11	9	Microwave Oven	5	1

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM)

DESKTOP-2

-- =====

-- TASK 5: GROUP BY + CASE

-- =====

-- Total revenue by store and category

```
SELECT
    s.store_name,
    c.category_name,
    SUM(oi.quantity* oi.unit_price) AS total_revenue
FROM [retailmart].[orders] o
JOIN [retailmart].[stores] s
ON s.store_id = o.store_id
JOIN [retailmart].[order_items] oi
ON o.order_id = oi.order_id
JOIN [retailmart].[products] p
ON p.product_id = oi.product_id
JOIN [retailmart].[categories] c
ON p.category_id = c.category_id
GROUP BY    s.store_name,c.category_name;
```


22-RetailMar...fujitsu (73))
1-RetailMart-...\fujitsu (72))

```

6      SELECT
7          s.store_name,
8          c.category_name,
9          SUM(oi.quantity * oi.unit_price) AS total_revenue
10     FROM retailmart.orders o
11    JOIN retailmart.stores s
12      ON s.store_id = o.store_id
13    JOIN retailmart.order_items oi
14      ON o.order_id = oi.order_id
15    JOIN retailmart.products p
16      ON p.product_id = oi.product_id
17    JOIN retailmart.categories c
18      ON p.category_id = c.category_id
19   GROUP BY
20      s.store_name,
21      c.category_name;
22

```

100 %
No issues found

Results

Messages

	store_name	category_name	total_revenue
1	RetailMart Karachi	Accessories	13500.00
2	RetailMart Lahore	Accessories	1800.00
3	RetailMart Karachi	Clothing	25500.00
4	RetailMart Lahore	Clothing	6600.00
5	RetailMart Karachi	Electronics	295000.00
6	RetailMart Lahore	Electronics	85000.00
7	RetailMart Islamabad	Groceries	3800.00
8	RetailMart Islamabad	Home Appliances	42400.00

Query executed successfully.
DESKTOP-26P7AE7 (17.0 F

-- Label each product as Budget / Mid-range / Premium using CASE

23-RetailMar...fujitsu (73) 1-RetailMart-...fujitsu (72)

```

5
6  -- Label each product as Budget / Mid-range / Premium using CASE
7
8  SELECT
9      product_id,
10     product_name,
11     price,
12
13     CASE
14         WHEN price < 2000 THEN 'Budget'
15         WHEN price BETWEEN 2000 AND 10000 THEN 'Mid-range'
16         ELSE 'Premium'
17     END AS price_category
18
19 FROM retailmart.products;
20
21 --SELECT * FROM retailmart.order_items;

```

100 % No issues found Ln: 8

Results Messages

	product_id	product_name	price	price_category
1	1	Samsung 4K TV	85000.00	Premium
2	2	iPhone 15	295000.00	Premium
3	3	Wireless Earbuds	4500.00	Mid-range
4	4	Leather Jacket	8500.00	Mid-range
5	5	Linen Shirt	2200.00	Mid-range
6	6	Rice 5kg	950.00	Budget
7	7	Cooking Oil 3L	750.00	Budget
8	8	Sunglasses	1800.00	Budget

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu...

-- =====

-- TASK 7: Subqueries

-- =====

-- Orders where the total exceeds the average order value (subquery in WHERE)

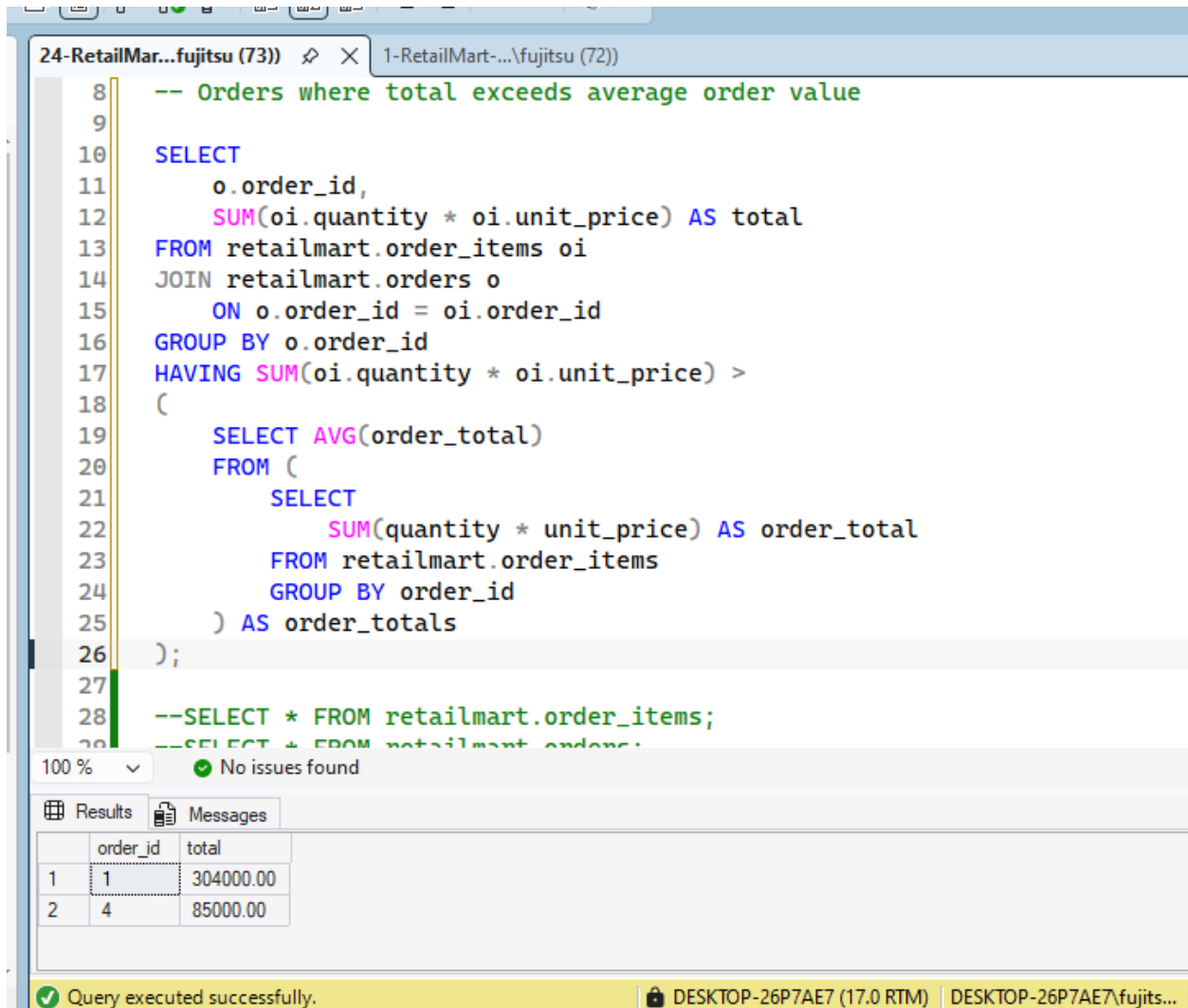
select o.order_id, sum([quantity]*[unit_price]) as total

from [retailmart].[order_items] o

group by o.order_id

having sum([quantity]*[unit_price]) >

(select avg([quantity]*[unit_price]) from [retailmart].[order_items]) --59,123



```
24-RetailMar...fujitsu (73) 1-RetailMart-...fujitsu (72)
8  -- Orders where total exceeds average order value
9
10 SELECT
11     o.order_id,
12     SUM(oi.quantity * oi.unit_price) AS total
13 FROM retailmart.order_items oi
14 JOIN retailmart.orders o
15     ON o.order_id = oi.order_id
16 GROUP BY o.order_id
17 HAVING SUM(oi.quantity * oi.unit_price) >
18 (
19     SELECT AVG(order_total)
20     FROM (
21         SELECT
22             SUM(quantity * unit_price) AS order_total
23         FROM retailmart.order_items
24         GROUP BY order_id
25     ) AS order_totals
26 );
27
28 --SELECT * FROM retailmart.order_items;
29 --SELECT * FROM retailmart.orders;
```

100 % No issues found

	order_id	total
1	1	304000.00
2	4	85000.00

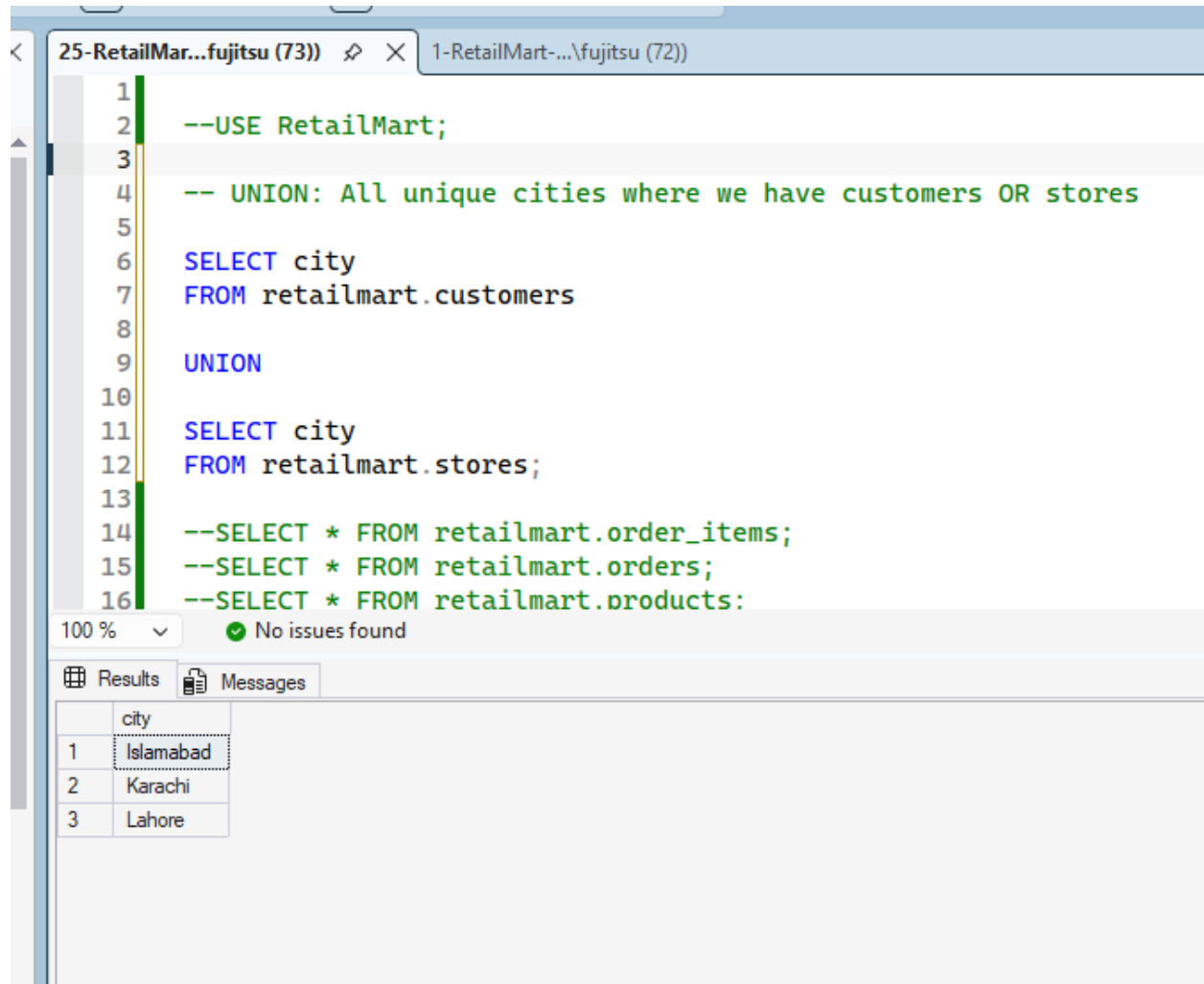
Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujits...

-- =====

-- TASK 8: Set operators — UNION, INTERSECT, EXCEPT

-- =====

-- UNION: All unique cities where we have customers OR stores



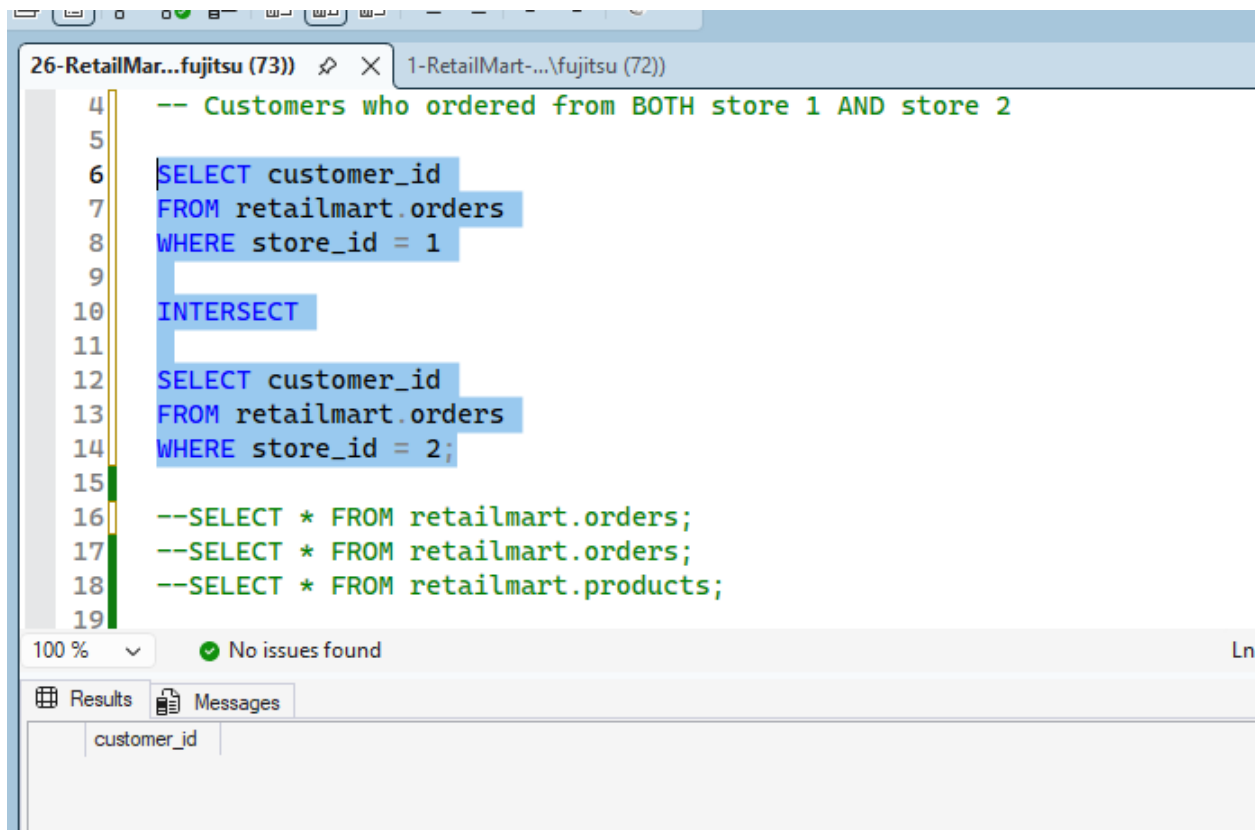
The screenshot shows a SQL IDE with two tabs: "25-RetailMar...fujitsu (73)" and "1-RetailMart-...fujitsu (72)". The active tab contains a SQL query. The query is as follows:

```
1
2  --USE RetailMart;
3
4  -- UNION: All unique cities where we have customers OR stores
5
6  SELECT city
7  FROM retailmart.customers
8
9  UNION
10
11 SELECT city
12 FROM retailmart.stores;
13
14 --SELECT * FROM retailmart.order_items;
15 --SELECT * FROM retailmart.orders;
16 --SELECT * FROM retailmart.products;
```

Below the query editor, there is a status bar showing "100 %" and "No issues found". At the bottom, there are two tabs: "Results" and "Messages". The "Results" tab is active, displaying a table with the following data:

	city
1	Islamabad
2	Karachi
3	Lahore

-- INTERSECT: Customers who ordered from BOTH store 1 AND store 2



```
26-RetailMar...fujitsu (73) 1-RetailMart-...fujitsu (72)
4  -- Customers who ordered from BOTH store 1 AND store 2
5
6  SELECT customer_id
7  FROM retailmart.orders
8  WHERE store_id = 1
9
10 INTERSECT
11
12 SELECT customer_id
13 FROM retailmart.orders
14 WHERE store_id = 2;
15
16 --SELECT * FROM retailmart.orders;
17 --SELECT * FROM retailmart.orders;
18 --SELECT * FROM retailmart.products;
19
```

100 % No issues found Ln

Results Messages

customer_id

-- EXCEPT: Customers who have NEVER placed an order

-- A - B A = {1,2,3,4} , B= {2,4,6,7}==== A-B = {1,3}

select customer_id from [retailmart].[customers] --A

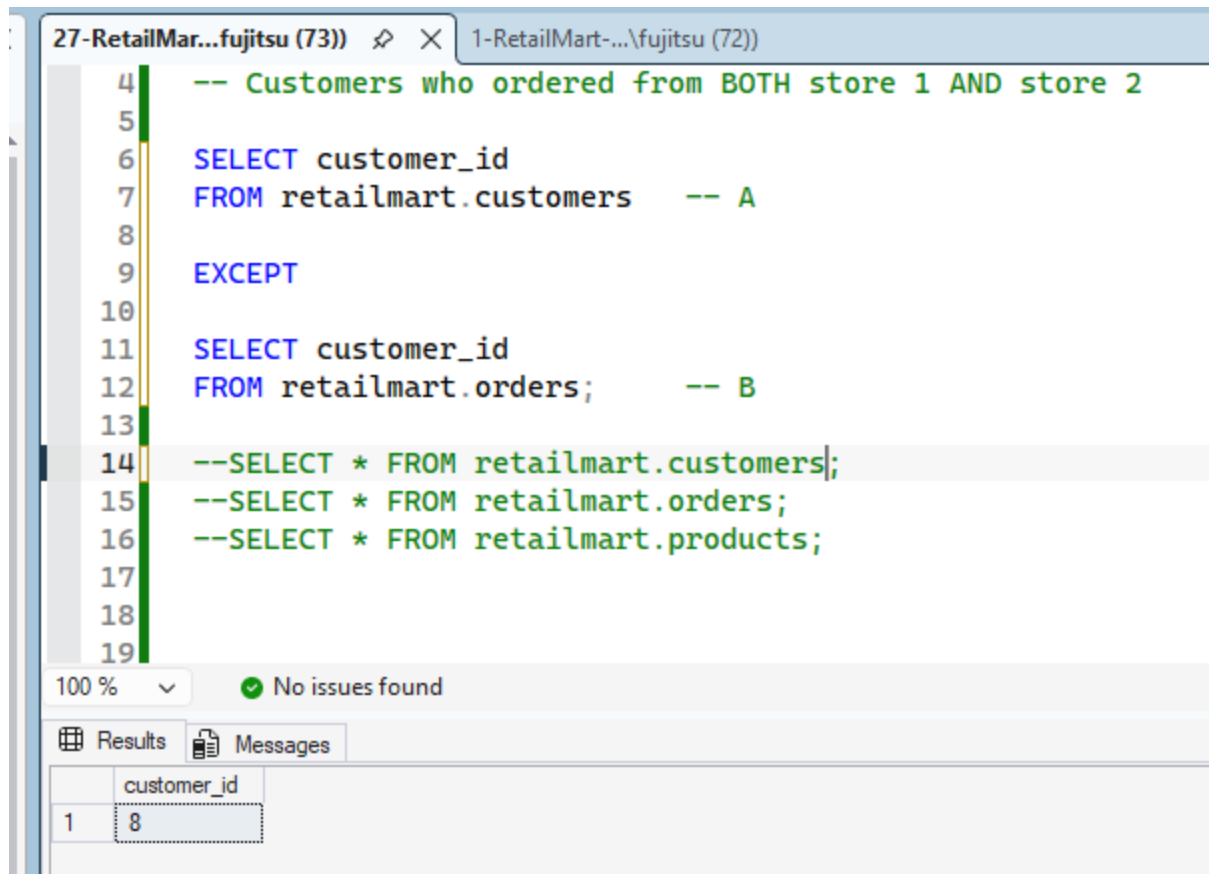
except

select customer_id from [retailmart].[orders] -- B

SELECT PRODUCT_ID FROM [retailmart].[products]

EXCEPT

SELECT PRODUCT_ID FROM [retailmart].[order_items]



-- =====

-- HOUR 3 — ADVANCED & AUTOMATE

-- =====

-- =====

-- TASK 9: CTEs + Recursive CTE

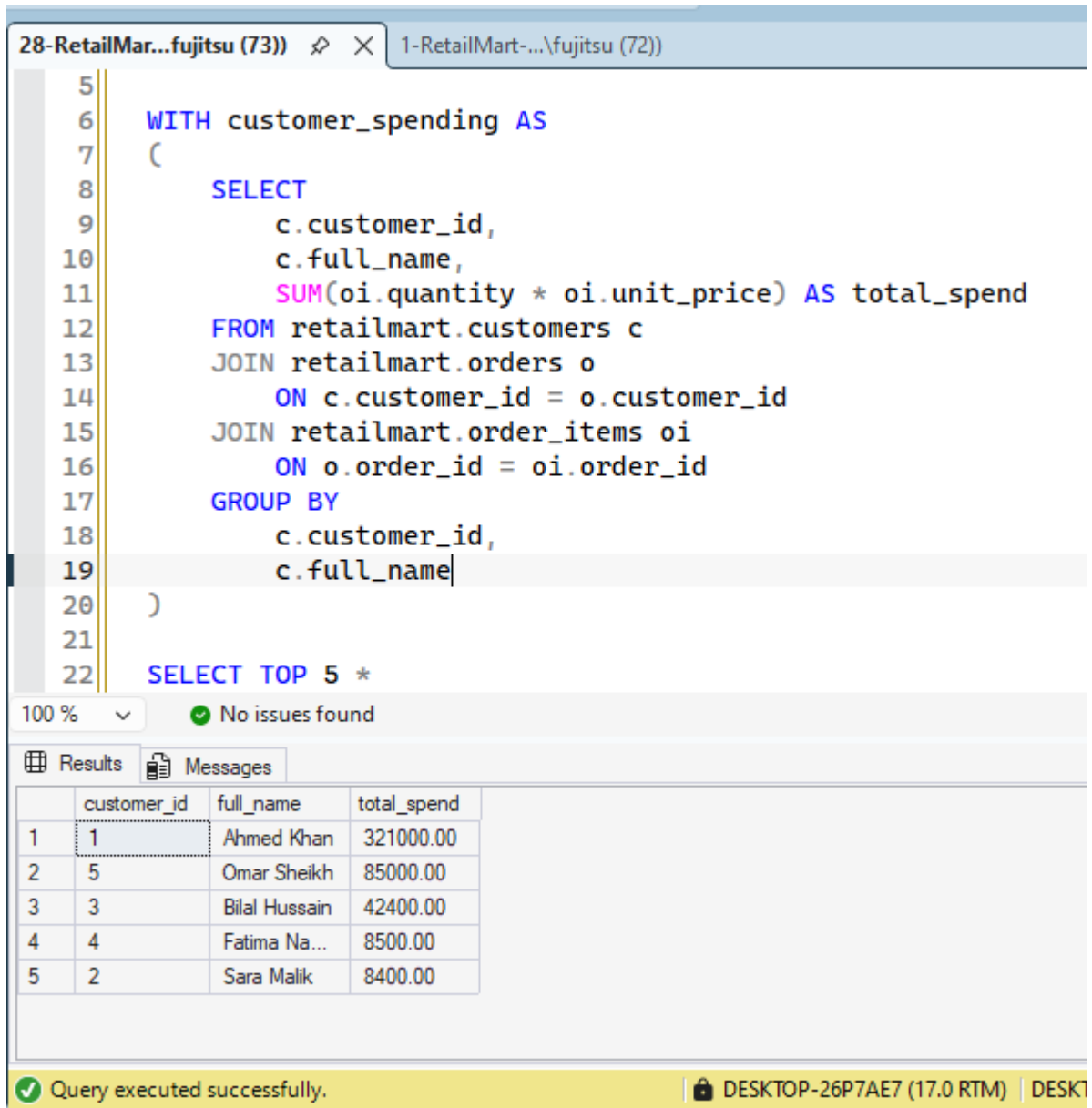
-- =====

-- Regular CTE: Top 5 customers by total spend

JOIN CUSTOMERS --- JOIN -- ORDERS -- JOIN - ORDER ITEMS --- QUANTITY * UNIT_PRICE

GROUP CUSTOMER_ID -- TOTAL SPENDING

TOP 5



```
5
6 WITH customer_spending AS
7 (
8     SELECT
9         c.customer_id,
10        c.full_name,
11        SUM(oi.quantity * oi.unit_price) AS total_spend
12    FROM retailmart.customers c
13    JOIN retailmart.orders o
14        ON c.customer_id = o.customer_id
15    JOIN retailmart.order_items oi
16        ON o.order_id = oi.order_id
17    GROUP BY
18        c.customer_id,
19        c.full_name
20 )
21
22 SELECT TOP 5 *
```

100 % No issues found

Results Messages

	customer_id	full_name	total_spend
1	1	Ahmed Khan	321000.00
2	5	Omar Sheikh	85000.00
3	3	Bilal Hussain	42400.00
4	4	Fatima Na...	8500.00
5	2	Sara Malik	8400.00

Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7

-- =====

-- TASK 10: Window functions

-- =====

-- ROW_NUMBER and RANK: Products ranked by revenue

```
CTE( PRODUCT -- JOIN -- ORDER ITEMS --
```

```
GROUP BY PRODUCT -- SUM(UNIT_PRICE* QUANTITY) -- REVENUE)
```

```
WITH CTE AS (
```

```
SELECT P.PRODUCT_ID, SUM(OI.UNIT_PRICE*OI.UNIT_PRICE) AS REVENUE
```

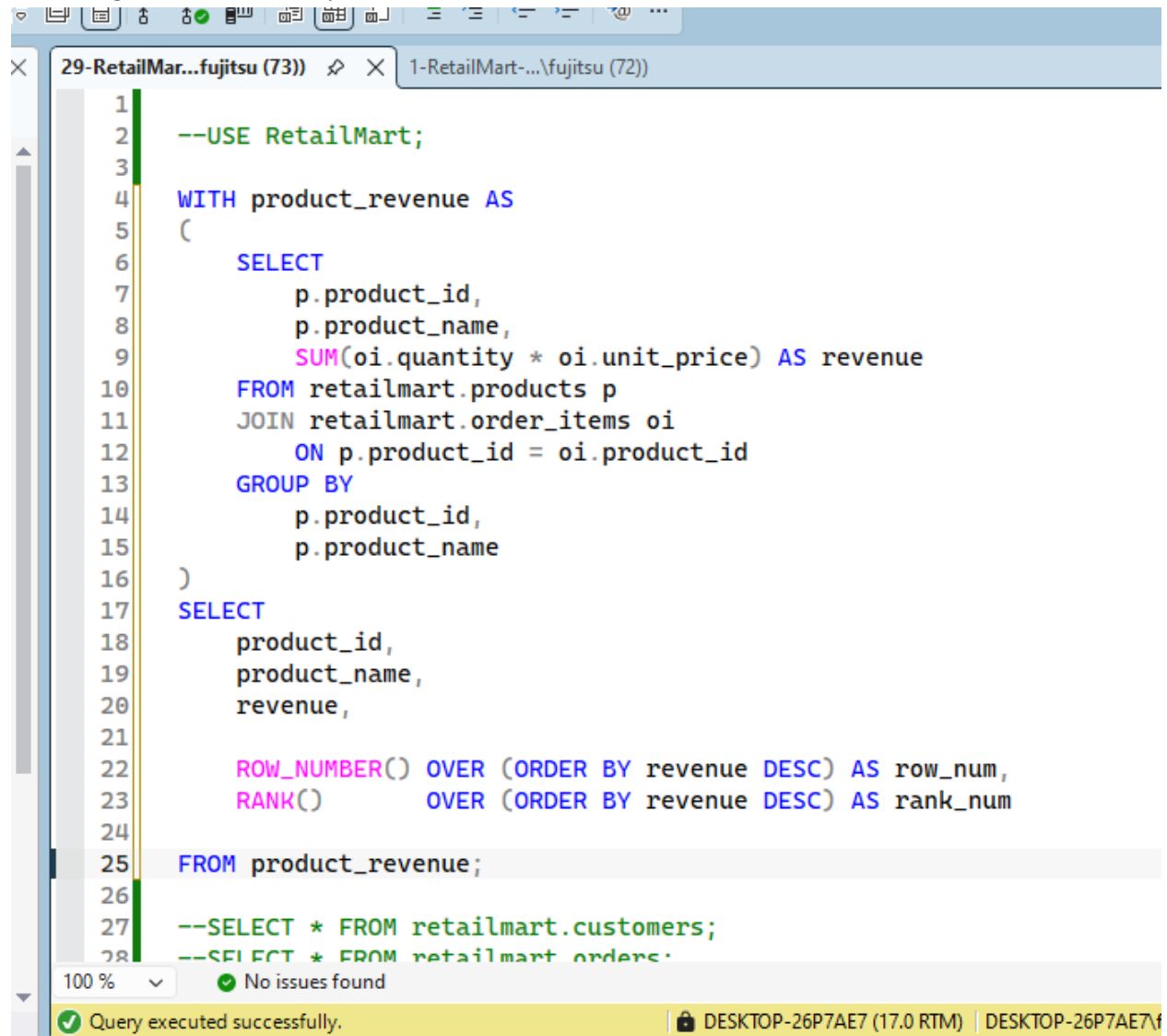
```
FROM [retailmart].[products] P INNER JOIN [retailmart].[order_items] OI
```

```
ON OI.product_id = P.product_id
```

```
GROUP BY P.PRODUCT_ID)
```

```
SELECT product_id, ROW_NUMBER() OVER(ORDER BY REVENUE DESC) AS RN FROM CTE
```


-- Running total of revenue by store



The screenshot shows a SQL IDE window with two tabs: '29-RetailMar...fujitsu (73)' and '1-RetailMart-...fujitsu (72)'. The active tab contains a SQL query. The query starts with a comment '--USE RetailMart;'. It then defines a CTE named 'product_revenue' which selects product_id, product_name, and the sum of (quantity * unit_price) as revenue from retailmart.products joined with retailmart.order_items on product_id. The CTE is then used in a main SELECT statement that selects product_id, product_name, revenue, ROW_NUMBER() OVER (ORDER BY revenue DESC) AS row_num, and RANK() OVER (ORDER BY revenue DESC) AS rank_num. The query ends with 'FROM product_revenue;'. Below the query, there are two commented-out lines: '--SELECT * FROM retailmart.customers;' and '--SELECT * FROM retailmart.orders;'. The status bar at the bottom indicates 'Query executed successfully.' and 'No issues found'.

```
1
2  --USE RetailMart;
3
4  WITH product_revenue AS
5  (
6      SELECT
7          p.product_id,
8          p.product_name,
9          SUM(oi.quantity * oi.unit_price) AS revenue
10     FROM retailmart.products p
11     JOIN retailmart.order_items oi
12         ON p.product_id = oi.product_id
13     GROUP BY
14         p.product_id,
15         p.product_name
16 )
17 SELECT
18     product_id,
19     product_name,
20     revenue,
21
22     ROW_NUMBER() OVER (ORDER BY revenue DESC) AS row_num,
23     RANK() OVER (ORDER BY revenue DESC) AS rank_num
24
25 FROM product_revenue;
26
27 --SELECT * FROM retailmart.customers;
28 --SELECT * FROM retailmart.orders;
```

100 % No issues found

Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\

29-RetailMar...fujitsu (73))
1-RetailMart-...\fujitsu (72))

```

1
2  --USE RetailMart;
3
4  WITH product_revenue AS
5  (
6      SELECT
7          p.product_id,
8          p.product_name,
9          SUM(oi.quantity * oi.unit_price) AS revenue
10     FROM retailmart.products p
11     JOIN retailmart.order_items oi
12         ON p.product_id = oi.product_id
13     GROUP BY
14         p.product_id,
15         p.product_name
16 )
17 SELECT
18     product_id,
19     product_name,

```

100 %
No issues found

Results

Messages

	product_id	product_name	revenue	row_num	rank_num
1	2	iPhone 15	295000.00	1	1
2	1	Samsung 4K TV	85000.00	2	2
3	9	Microwave Oven	36000.00	3	3
4	4	Leather Jacket	25500.00	4	4
5	3	Wireless Earbuds	13500.00	5	5
6	5	Linen Shirt	6600.00	6	6

Query executed successfully.
DESKTOP-26P7AE7 (17.0 RTM) | DESK

-- =====

-- TASK 12: Views

-- =====

-- Standard view: Order summary

-- Use the view

-- Indexed (materialized) view: Store revenue totals

-- Note: Requires SCHEMABINDING + unique clustered index

30-RetailMart...fujitsu (73))* 1-RetailMart-...fujitsu (72))

```
4  -- Standard View: Order summary
5
6  CREATE VIEW retailmart.v_order_summary
7  AS
8  SELECT
9      o.order_id,
10     o.order_date,
11     c.customer_id,
12     c.full_name,
13     SUM(oi.quantity * oi.unit_price) AS order_total
14 FROM retailmart.orders o
15 JOIN retailmart.customers c
16     ON o.customer_id = c.customer_id
17 JOIN retailmart.order_items oi
18     ON o.order_id = oi.order_id
19 GROUP BY
20     o.order_id,
21     o.order_date,
22     c.customer_id,
23     c.full_name;
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-06-07T11:50:17.4946528+05:00

100 % No issues found

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7 (17.0 RTM)

- Database Diagrams
- Tables
- Views
- System Views
- retailmart.v_order_summary
- Dropped Ledger Views
- External Resources
- Synonyms

-- =====

-- TASK 13: Indexes

-- =====

-- Composite index: Speed up queries filtering by customer + date

-- Filtered index: Only index Pending orders (partial index)

-- Non-clustered index on customer email for fast lookups

-- Verify indexes

EXEC sp_helpindex 'retailmart.orders';

EXEC sp_helpindex 'retailmart.customers';

31-RetailMar...ujitsu (73))* 1-RetailMart-...fujitsu (72))

```

14
15 SELECT *
16 FROM retailmart.orders
17 WHERE status = 'Pending';
18 -- Non-clustered index on email
19
20 CREATE NONCLUSTERED INDEX IX_customers_email
21 ON retailmart.customers (email);
22
23
24 EXEC sp_helpindex 'retailmart.orders';
25 EXEC sp_helpindex 'retailmart.customers';
26 --SELECT * FROM retailmart.customers;
27 --SELECT * FROM retailmart.orders;
28 --SELECT * FROM retailmart.products;|
29
30
31
32
33

```

100 % 2 0

Results Messages

	index_name	index_description	index_keys
1	IX_customers_email	nonclustered located on PRIMARY	email
2	PK__customer__CD65CB855F455E43	clustered, unique, primary key located on PRIMARY	customer_id
3	UQ__customer__AB6E6164A401B853	nonclustered, unique, unique key located on PRIMA...	email

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7

-- =====

-- TASK 14: Stored Procedures + Error Handling

-- =====

-- Procedure 1: Monthly sales report for a store

-- Test the report for Store 1 (Karachi)

EXEC retailmart.sp_MonthlySalesReport @StoreID = 1, @Year = 2024;

-- Test with default year

EXEC retailmart.sp_MonthlySalesReport @StoreID = 2;

32-RetailMar...ujitsu (73))*

1-RetailMart-...\fujitsu (72))

```
2  --USE RetailMart;
3
4  CREATE PROCEDURE retailmart.sp_MonthlySalesReport
5      @StoreID INT,
6      @Year INT = NULL  -- default year
7  AS
8  BEGIN
9      SET NOCOUNT ON;
10
11     BEGIN TRY
12
13         -- If year not provided, use current year
14         IF @Year IS NULL
15             SET @Year = YEAR(GETDATE());
16
17         SELECT
18             s.store_name,
19             MONTH(o.order_date) AS sale_month,
20             SUM(oi.quantity * oi.unit_price) AS total_sales
21         FROM retailmart.orders o
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-06-07T12:04:57.7904575+05:00

100 % No issues found

Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\


```
12
13      -- If year not provided, use current year
14      IF @Year IS NULL
15          SET @Year = YEAR(GETDATE());
16
17      SELECT
18          s.store_name,
19          MONTH(o.order_date) AS sale_month,
20          SUM(oi.quantity * oi.unit_price) AS total_sales
21      FROM retailmart.orders o
22      JOIN retailmart.stores s
23          ON o.store_id = s.store_id
24      JOIN retailmart.order_items oi
25          ON o.order_id = oi.order_id
26      WHERE
27          o.store_id = @StoreID
28          AND YEAR(o.order_date) = @Year
29      GROUP BY
30          s.store_name,
31          MONTH(o.order_date)
32      ORDER BY
33          sale_month;
34
35      END TRY
36      BEGIN CATCH
37          SELECT
38              ERROR_NUMBER() AS error_no,
39              ERROR_MESSAGE() AS error_message.
```

100 % No issues found

✓ Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fuj

32-RetailMar...ujitsu (73))* 1-RetailMart-...fujitsu (72))

```

24      JOIN retailmart.order_items oi
25      ON o.order_id = oi.order_id
26  WHERE
27      o.store_id = @StoreID
28      AND YEAR(o.order_date) = @Year
29  GROUP BY
30      s.store_name,
31      MONTH(o.order_date)
32  ORDER BY
33      sale_month;
34
35  END TRY
36  BEGIN CATCH
37      SELECT
38          ERROR_NUMBER() AS error_no,
39          ERROR_MESSAGE() AS error_message;
40  END CATCH
41  END;
42
43  --Test the Procedure

```

100 % No issues found

Results Messages

	store_name	sale_month	total_sales
1	RetailMart Karachi	1	312500.00
2	RetailMart Karachi	4	4500.00
3	RetailMart Karachi	5	17000.00

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7/f

```

41  END;
42
43  --Test the Procedure
44  EXEC retailmart.sp_MonthlySalesReport
45      @StoreID = 1,
46      @Year = 2024;
47
48
49
50

```

-- =====

-- END OF CAPSTONE PROJECT

```
-- =====  
  
-- You have covered:  
  
-- DDL      CREATE TABLE, constraints, data types  
  
-- DML      INSERT, UPDATE, DELETE, MERGE  
  
-- Querying  SELECT, WHERE, ORDER BY  
  
-- Joins     INNER, LEFT, RIGHT  
  
-- Aggregation GROUP BY, ROLLUP, CUBE, GROUPING SETS  
  
-- CASE      Price tier labelling  
  
-- Subqueries Inline  
  
-- Set ops   UNION, INTERSECT, EXCEPT  
  
-- CTEs      Regular  
  
-- Windows   ROW_NUMBER, RANK  
  
-- PIVOT     Monthly revenue matrix  
  
-- Views     Standard + Indexed  
  
-- Indexes   Composite, Filtered, Non-clustered  
  
-- Procedures Parameters, defaults  
  
-- =====
```